

INTERVIEW PREP · A NEW FORMAT

AI-Enabled Coding Interviews

How to pass the newest kind of coding round — where you're given an AI assistant and a real, multi-file codebase, and judged on your *judgment and control*, not your typing speed.

Used at Meta, LinkedIn, Uber, Shopify, Canva, Rippling and more.

What this guide is

A growing number of companies have replaced (or added) a coding round where you're **allowed — even expected — to use an AI assistant**. This is not "cheating"; it's the new job. The problems are bigger than classic LeetCode: you might be dropped into a **multi-file codebase** and asked to fix bugs, add a feature, and optimize it, all in ~45 minutes, with an AI at your side.

The catch: because the AI can write the code, the interview is no longer testing whether you *can* write it. It's testing something harder to fake — **do you know what to build, can you direct the AI to build it, and can you tell when it's wrong?**

The one-sentence thesis

Use the AI for **volume** (boilerplate, standard patterns, tests) and yourself for **judgment** (the approach, the data structures, the edge cases, spotting bad output). The candidates who pass are the ones visibly *in control* of the AI — not the ones who paste the prompt and hope.

What's inside

Part 1 — The landscape 1. **The two interview formats** — structured (their tools) vs. open-ended (your tools) 2. **The seven patterns** — the algorithmic toolkit you still must recognize

Part 2 — The core skills (the interview is really testing these) 3. **Codebase orientation** — the first 5–10 minutes in an unfamiliar repo 4. **Planning your approach** — decompose before you prompt 5. **Driving the AI** — prompting at the right altitude, and when to code yourself 6. **Verification & testing** — proving it works, and catching the AI's mistakes 7. **Communication** — narrating so the interviewer sees your judgment

Part 3 — Putting it together 8. **A worked session** — a full annotated walk-through of a real-style problem

How to read this

Green boxes restate an idea simply, blue boxes are concrete examples, and red

Remember this boxes hold the one line worth keeping from each chapter. Chapters are short by design.

Remember this

The AI writes the code; you supply the judgment. Every chapter here is about making that judgment **visible and reliable** under interview pressure.

1. The two interview formats

Before you prepare, find out which of two formats you're walking into — they demand different things. **Ask your recruiter directly**; this is a fair, normal question.

Format A — Structured (you use their environment)

The company gives you a browser-based coding environment with a built-in AI assistant — typically **CoderPad** (Meta, LinkedIn) or **HackerRank** (Uber). The problem usually follows a predictable three-phase arc:

PHASE 1		PHASE 2		PHASE 3
Fix the bug (understand the codebase)	->	Add a feature (build on the existing code)	->	Optimize it (make it faster / handle more scale)

- You're often dropped into an **existing multi-file codebase** and asked to navigate it.
- The AI quality is **whatever they provide** — you can't swap in your favorite tool.
- Good news: **Meta and LinkedIn usually offer a practice session** so you can learn the platform's quirks beforehand. Take it.

Format B — Open-ended (you bring your own environment)

You use **your own IDE and your own AI tools** while screen-sharing. Common at **Shopify, Canva, Rippling**. Problems are usually **greenfield** — build something from scratch — rather than navigating existing code.

- Advantage: you work in tools you already know.
- The catch, in the guide's own words: *"your workflow is completely visible, which means it needs to actually be good."* There's nowhere to hide a sloppy process.

Why the format changes your prep

Structured = you're **adapting to their constraints**, so practice navigating unfamiliar code and the bug→feature→optimize arc. Open-ended = you're **showcasing your natural workflow**, so rehearse your own setup until your prompting, file navigation, and testing look smooth on a shared screen.

Questions to ask your recruiter

Send these before the interview — the answers reshape how you prepare:

1. Is it **structured (your platform)** or **open-ended (my own IDE/tools)**?
2. Which **platform and AI assistant** will be used?
3. Is there a **practice environment** I can try first?
4. Are there **tool restrictions** (allowed/blocked AI tools, internet access)?
5. Is it **greenfield** or an **existing codebase** I'll navigate?

Remember this

There are two formats — their tools (structured, often bug→feature→optimize on an existing codebase) or your tools (open-ended, usually greenfield). Ask the recruiter which one, and prepare for that one specifically.

2. The seven patterns

Here's the twist that surprises people: **the AI writes the code, but you still have to know the algorithm**. The interview tests whether you can look at a problem, recognize *which* approach fits, and then direct the AI toward it. The guide puts it perfectly:

"The AI handles the implementation. What you bring is the judgment behind it."

So the classic patterns from your DSA prep don't go away — they become the **language you use to command the AI**. You won't hand-write Dijkstra, but you must recognize "this is a shortest-path problem" and say so.

The seven core patterns

#	Pattern	You reach for it when...
1	Graph Search & Pathfinding	navigating a network / finding a route — BFS, DFS, Dijkstra, A*
2	Topological Sort	ordering things with dependencies — tasks, build steps, course prereqs
3	Backtracking	constraint problems — puzzles, generating valid configurations
4	Greedy & Bin Packing	allocation/scheduling where a local best choice works
5	Dynamic Programming	overlapping subproblems — "count the ways / best value"
6	String Matching & Parsing	text patterns, regex, evaluating expressions
7	Data Structure Design	compose maps/heaps/queues into a custom structure for given performance needs

The mistake to avoid

The single most common failure, per the guide:

"One mistake candidates make constantly is having a vague (wrong) idea of how to solve a problem."

A vague idea produces vague prompts, which produce wrong code — and now you're debugging the AI's version of *your* confusion. **Clarity first, prompt second.** If you can't state the approach in one clear sentence, you're not ready to prompt yet.

Recognize, then command

Problem: *"Given package dependencies, find a valid install order."* You recognize **topological sort**, so you tell the AI exactly that: *"Build a directed graph of packages, then return a topological ordering using Kahn's algorithm; detect cycles and report them."* That's a world apart from *"figure out the install order."*

If you're short on prep time

The guide's triage: prioritize **Graph Search, Topological Sort, and Backtracking** — they cover most current AI-coding problems. (These map directly to your DSA Vol 1 graphs/backtracking and Vol 3/4 advanced-graphs material — the algorithms are the same; only your *role* changes.)

Remember this

You still must recognize the pattern — the seven above cover most problems. The AI writes it; you name it. A precise, correctly-recognized approach is the whole game, because a vague idea becomes a vague prompt becomes wrong code.

3. Codebase orientation

In a structured interview you're often dropped into an **unfamiliar multi-file codebase**. The instinct under time pressure is to start prompting immediately. **Resist it.** Jumping straight to prompting without understanding the code signals poor judgment — and it leads to solutions that fight the existing patterns.

Spend the **first 5–10 minutes** orienting. It feels slow; it pays for itself many times over and it reads as professionalism.

The seven-step orientation

1. ENTRY POINTS	where does execution start? (main, API handler, CLI)
2. KEY FUNCTIONS	the "hard core" of complex methods everything builds on
3. DATA MODELS	how objects relate: Game -> Board -> Cell
4. ARCHITECTURE	what does the directory layout tell you? (MVC? layers? features?)
5. PUBLIC INTERFACE	which methods are public vs private (how you'll call things)
6. STATE	where does state live, how does it change?
7. TESTS	run them: they document intended behavior AND verify the baseline

1. **Find entry points.** Locate where execution begins and trace the data flow outward — that reveals how the pieces talk to each other.
2. **Identify key functions.** Most codebases have a small "hard core" of complex functions; understanding those gives you disproportionate insight.
3. **Map class hierarchies and data models.** Knowing that a `Game` contains a `Board` which contains `Cell`s teaches you more than reading any single method body.
4. **Recognize architectural patterns.** The directory structure reveals the organizing philosophy — models/views/controllers, layers, or feature folders. Follow the existing pattern; don't invent a parallel one.
5. **Study public interfaces.** Knowing what's public vs private tells you how to instruct the AI to interact with existing classes.
6. **Understand state management.** Single source of truth vs. distributed state creates very different bug patterns — know which you're dealing with.
7. **Review the tests.** Tests document *intended* behavior and give you a verification mechanism. **Run them first** to confirm the baseline works before you change anything.

Use the AI to orient — the right way

The AI is a great orientation accelerator, but use it well:

- **Ask for inline comments** explaining each class/method, rather than a chat Q&A. This keeps your eyes on the *actual* code while it explains — you're reading, not just listening.
- **Always verify the AI's explanation against the source.** The AI can misread purpose, miss side effects, or describe *intended* rather than *actual* behavior. Trust, then check.

What good orientation sounds like

"Before I change anything, let me get oriented. `main.py` is the entry point → it constructs a `Scheduler` that owns a `TaskQueue`. The core logic is in `Scheduler.run()`. State lives in one place, the queue. Let me run the tests... okay, 11 pass, 2 fail — those 2 failures are probably the bug I'm here to fix. Now I understand the shape, let me plan." That 90 seconds of narration just showed the interviewer more senior judgment than any prompt could.

Remember this

Spend the first 5–10 minutes orienting, not prompting: entry points, key functions, data models, architecture, interfaces, state, and **run the tests**. Ask the AI for inline comments to speed it up — then verify what it tells you against the real code.

4. Planning your approach

Once you understand the code and know the pattern, **plan before you prompt**. A plan turns one impossible prompt ("solve this") into a sequence of achievable ones — and stated aloud, it lets the interviewer redirect you before you waste time.

Two ways to plan (ask which they prefer)

1. **AI drafts, you vet.** Ask the AI for a plan, then critically review it before executing.
2. **You sketch, AI implements.** Design the architecture yourself, then bring the AI in for each piece.

Neither is wrong — **ask the interviewer which they'd like to see**. It shows you know the choice exists.

Decompose into 3–5 phases

Break the problem into discrete, sequential steps. The right granularity is a step you could **explain on its own** — not too broad, not too narrow:

T00 BROAD:	"Build the scheduler"	(a whole feature — unclear, unverifiable)
JUST RIGHT:	"Add a ready-queue and pop tasks whose deps are met"	(one clear, testable step)
T00 NARROW:	"Write the Task constructor"	(trivial — micromanaging the AI)

A strong decomposition usually orders itself like this:

1. **Data models first** — define the structures before the logic that uses them.
2. **Core functionality before edge cases** — get the happy path solid, then handle corners.
3. **Bug fixes before dependent features** — don't build on broken code.

Say the plan out loud

State your plan before you write anything. It takes seconds and it:

- **demonstrates strategic thinking** (the thing being graded),
- **lets the interviewer redirect you** early if you're off,
- **creates shared understanding** so your later steps make sense.

Turn the plan into prompts

Each phase becomes a specific, codebase-aware prompt. Vague in, vague out:

```
WEAK: "solve this problem"
STRONG: "Implement a BFS traversal of the maze, starting from the start
        position in the Grid class, returning the shortest path as a
        list of coordinates."
```

Notice the strong prompt names the **real class** (`Grid`), the **algorithm** (BFS), and the **exact return shape**. That comes straight from your orientation + pattern recognition.

Revise the plan — but on your terms

Plans should evolve with new information (a performance constraint, a hint, messy data). But **don't abandon your approach just because the AI suggests an alternative**. If you pivot, you must be able to say *why* the new way is better. The interviewer is watching who's in charge.

A plan stated in ten seconds

"My plan: first define the `Task` and `Graph` data models, then build the dependency graph, then run a topological sort for the ordering, and finally handle the cycle-detection edge case. I'll verify after each step. Sound good, or would you rather I start somewhere else?"

Remember this

Decompose into 3–5 explainable phases (data models → core → edge cases), say the plan aloud before coding, and turn each phase into a specific, codebase-aware prompt. Evolve the plan with new facts — but only pivot when *you* can explain why.

5. Driving the AI

This is the heart of the interview. Anyone can paste a prompt; the signal the interviewer is reading is whether you **drive** the AI with control and judgment.

Prompt at the right altitude

The failure modes sit at two extremes:

T00 VAGUE	<----- JUST RIGHT ----->	T00 MICRO
"fix this"	clear objective +	dictating every
(AI guesses,	approach + constraints,	line (you're just
often wrong)	AI fills in implementation	typing through the AI)

- For **complex logic**: be precise about the **approach, data structures, and constraints** so the AI builds what you actually want.
- For **simple tasks**: stay loose and let it fill in the details.
- Always **reference the real codebase** — actual class names, method signatures, existing structures. This makes the generated code integrate naturally *and* signals to the interviewer that you genuinely understood the system during orientation.

Delegate vs. do it yourself

Not everything should go to the AI. The rule of thumb:

Situation	Do what
Quick fix, ~30 seconds of typing	Do it yourself — shows you can actually code
5+ minutes of standard-pattern work, boilerplate, tests	Delegate to the AI
A judgment call (approach, data structure, edge cases)	You own it — never delegate

The core principle, in the guide's words: **"use AI for volume and yourself for judgment."** Let the AI handle speed-critical, easily-verifiable chunks; you keep the decisions.

Keep the code healthy: verify after each generation

Run and verify after every meaningful AI generation. Don't stack three changes and then compile — conflicts create cascading errors that are hard to untangle. Compiling between steps also gives you natural checkpoints to narrate progress to the interviewer.

When the AI produces bad output

AI output has predictable failure patterns — watch for them:

- **Training-data bias** — the textbook solution that ignores *your* specific constraints.
- **Excessive verbosity** — walls of code where a few lines would do.
- **Over-engineering** — abstractions and patterns the problem never asked for.
- **Dangerous shortcuts** — deleting a failing test, coercing a type, or `catch {}`-ing an error to make red turn green. **Never let this slide** — catching it is a huge positive signal.

The recovery rule keeps you from wasting time:

```
Bad output?  -> RE-PROMPT ONCE with more specificity (don't just repeat yourself)
              -> still wrong? -> WRITE IT YOURSELF
Never spend more than ~2 minutes fighting a single generation.
```

Driving, out loud

"That works but it's over-engineered — it added a whole factory class we don't need. Let me re-prompt: 'Simplify to a single function that takes the grid and returns the path; no new classes, match the existing style in `Solver.java`.'" — one specific re-prompt, and if it's still wrong next time, you write it. That's control.

Catching a dangerous shortcut

The AI makes a failing test pass by... deleting the assertion. You catch it: *"It removed the test instead of fixing the bug — that's not a real fix. Let me restore the test and re-prompt to fix the actual logic."* Interviewers love this. It's the whole point of the round.

Remember this

Prompt at the right altitude (clear objective + approach + constraints, referencing real class names), delegate volume but keep judgment, compile after every generation, and follow the **one re-prompt, then do it yourself** rule. Spotting the AI's bad shortcuts is a top positive signal — never wave them through.

6. Verification & testing

AI-generated code that *looks* right is the trap of this whole format. The AI is fluent, so its output is plausible — and plausible is not the same as correct. **Verification is where you earn the offer**, because it's the habit that separates someone who trusts the AI blindly from someone who ships working software.

Verify continuously, not at the end

Don't write the whole solution and test once at the finish. **Run after each meaningful generation** (this is the same checkpoint rule from Chapter 5). Continuous verification:

- catches errors while they're small and isolated (not tangled with three later changes),
- gives you natural moments to narrate progress,
- means you're never more than one step away from known-good code.

What to actually run

1. RUN THE EXISTING TESTS first, to establish the baseline works
2. AFTER EACH CHANGE: compile + run the relevant tests again
3. ADD TESTS for new behavior (great task to delegate to the AI — then read them!)
4. TRACE EDGE CASES by hand empty input, one element, duplicates, cycles, overflow, the boundary the problem hinges on
5. CONFIRM against the problem does the output actually match what was asked?

Edge cases are your judgment on display

The AI will usually nail the happy path. The **edge cases are where you add value** — and where interviewers look hardest. Before you call something done, ask:

- Empty / null input? A single element?
- Duplicates, ties, or repeated values?
- Cycles (in graphs), overflow (in sums/products), off-by-one at boundaries?
- The specific corner the problem is *really* about (a maze with no exit, a schedule with a circular dependency, a full board)?

Prompt the AI to test these explicitly — *and read the tests it writes*, because a test that asserts the wrong thing is worse than no test.

Don't trust an AI test you didn't read

A subtle failure: the AI writes tests that pass but assert incorrect behavior, or that quietly match a bug. **A green checkmark you didn't verify is not evidence.** Read every test's assertion and confirm it encodes what the problem actually requires.

Verification, out loud

"It compiles and the 3 sample cases pass. Now the edge cases: empty grid — let me add a test... returns empty path, good. A maze with no exit — hmm, it loops forever; that's a real bug. Let me re-prompt to return `null` when the queue empties without reaching the target." That sequence — happy path, then edges, then a caught bug — is exactly the rigor being graded.

Remember this

Plausible $\neq$ correct. Run the tests first for a baseline, verify after every change, and hunt the edge cases yourself — empty, single, duplicate, cycle, overflow, boundary. And never trust an AI-written test you haven't read: a green check you didn't verify isn't evidence.

7. Communication

Here's the thing that's easy to miss: in an AI-enabled interview, **the code is no longer the signal — your thinking is**. The AI can produce working code for almost anyone. What distinguishes you is *visible judgment*: why you chose this approach, how you caught that bug, when you overrode the AI. If you work in silence, the interviewer sees a code generator, not an engineer.

Narrate the judgment, not the typing

You don't need to read every line aloud. Narrate the **decisions**:

NARRATE THIS

- why you chose the approach
- what you learned in orientation
- why you're re-prompting
- a bug you spotted and why
- the edge cases you're checking
- a tradeoff you're weighing

SKIP THIS

- "now I'm typing a for loop"
- reading boilerplate verbatim
- every keystroke
- obvious syntax
- trivial getters/setters

Talk through the whole arc

Map your narration to the phases from earlier chapters:

1. **Orientation**: *"Let me get oriented before I touch anything..."* (Chapter 3)
2. **Plan**: *"Here's my plan — data models, then the graph, then topo sort. Sound good?"* (Chapter 4)
3. **Driving**: *"I'll delegate the boilerplate but I want to own the traversal logic..."* (Chapter 5)
4. **Verifying**: *"Compiles — now the edge cases: empty, single, cycle..."* (Chapter 6)
5. **Wrap**: *"It's correct and handles the edges; if I had more time I'd optimize the repeated lookups with a cache."*

Treat hints as gifts

If the interviewer nudges you, they're helping — take it. *"Good point, let me reconsider..."* beats defending a shaky choice. And when you disagree with an AI suggestion or a hint, **say why** — a reasoned override is a strong signal; stubborn silence is not.

The meta-signal

Everything in this guide ladders up to one question the interviewer is silently asking: **"Do I want this person driving AI tools on my team when it matters?"** They're looking for someone who plans, stays in control, verifies, catches the AI's mistakes, and explains their reasoning. Show them that person.

Silence vs. narration — same code, different outcome

Two candidates produce identical working solutions. One typed quietly and said "done." The other said: *"I recognized this as topological sort, planned it in four steps, delegated the graph-building but wrote the cycle check myself because that's the tricky part, and I caught the AI trying to skip the cycle case — added a test for it."* Only the second one demonstrated the judgment the round exists to measure.

Remember this

The code isn't the signal — your judgment is, and it only counts if it's visible. Narrate decisions (not keystrokes): approach, re-prompts, caught bugs, edge cases. Treat hints as gifts, and when you override the AI, say why. Show them the person they'd want driving AI on their team.

8. A worked session (annotated)

Here's a full, realistic structured session — the **bug** → **feature** → **optimize** arc — with the candidate's words and the reasoning behind each move. Read it out loud; this is the rhythm to internalize. [C] = candidate (spoken), [»] = annotation.

The setup. "You're given a `maze-solver` repo. There's a failing test. Fix the bug, then add a feature to return the shortest path, then make it handle large mazes. You have an AI assistant. ~45 minutes."

Minutes 0–8 — Orientation (Chapter 3)

[C] "Before I change anything, let me get oriented." [C] "`main.py` is the entry point → builds a `Grid` from input, calls `Solver.solve()`. The core is `Solver.solve()`. Data model: `Grid` holds a 2D list of `Cell`s, each with `isWall` and coordinates. State lives in the `Grid` — single source of truth, good." [C] "Let me ask the AI to add inline comments to `Solver` so I read it faster... okay, and I'm checking those comments against the actual code — yes, `solve()` currently does a DFS." [C] "Running the tests first: 5 pass, 1 fails — `test_finds_exit` expects it to reach the exit but it returns 'no path'. That's my bug."

[»] Notice: no prompting yet. Baseline established, bug located, shape understood. Cost so far: ~7 minutes. Worth every second.

Minutes 8–18 — Phase 1: Fix the bug (Chapters 4–6)

[C] "My hypothesis: the DFS isn't marking cells visited, so it loops or gives up early. Let me confirm by reading `solve()` ... right — no visited set. That's the bug." [C] "This is a small, well-understood fix, so I'll prompt specifically: 'In `Solver.solve()`, add a visited set so the DFS doesn't revisit cells; use `(row, col)` tuples, matching the existing coordinate style.'" [C] "Generated. Let me compile and run... `test_finds_exit` passes now, all 6 green." [C] "Edge check before I move on: a maze with no exit — let me add that test... good, it returns 'no path' instead of looping."

[»] Recognized the pattern (graph traversal needs visited), wrote a precise codebase-aware prompt, verified immediately, added an edge case. Textbook.

Minutes 18–33 — Phase 2: Add shortest-path (Chapters 2, 4, 5)

[C] "Shortest path in an unweighted grid → that's BFS, not DFS. DFS finds a path; BFS finds the shortest. Let me state my plan: switch the traversal to BFS with a queue, track each cell's parent, then reconstruct the path by walking parents back from the exit. I'll keep the visited set. Sound good?" [C] "I'll delegate the BFS boilerplate but I want to own the path-reconstruction, since that's the fiddly part: 'Implement BFS in `Solver.solve()` from the start cell, using a queue and a parent map keyed by `(row,col)` ; return the shortest path as a list of coordinates from start to exit.'" [C] "Compiles. Sample maze returns a path of length 9 — let me hand-trace... yes, that's shortest. Now edge cases: `start == exit` (returns just the start — good), no exit (returns empty — good)."

[>] The key senior move: recognizing BFS-not-DFS and saying WHY (shortest vs any). Delegated volume, kept the tricky judgment part, verified with a hand trace.

Minutes 33–42 — Phase 3: Optimize (Chapters 5, 6)

[C] "For large mazes, the concern is the visited structure and re-scanning. Let me ask the AI: 'Profile-wise, what's the bottleneck in this BFS for a 1000x1000 grid?'... it says the parent map of tuples is memory-heavy." [C] "It suggests rewriting everything with a fancy custom class — that's over-engineered for this. I'll re-prompt once, more specifically: 'Keep the BFS; just store visited as a 2D boolean array instead of a hash set of tuples, matching the `Grid` dimensions.'" [C] "Better — same result, less overhead. Compiles, all tests still green."

[>] Caught the AI over-engineering, applied the "one specific re-prompt" rule, and kept control of the design instead of accepting the fancy rewrite.

Minutes 42–45 — Wrap (Chapter 7)

[C] "To summarize: I fixed the missing-visited bug, switched DFS→BFS to guarantee the shortest path with parent-pointer reconstruction, and optimized the visited structure to a 2D array for large grids. All tests pass including the no-exit and `start==exit` edges. With more time I'd add A with a Manhattan-distance heuristic to prune the search on very large mazes."*

[>] A crisp recap that maps every phase to a decision — and a forward-looking optimization (A*) that shows range without over-building now.

What made this a pass

Every principle from the guide showed up, *visibly*:

Chapter	The moment it appeared
3 Orientation	7 minutes reading + running tests before any prompt
2 Patterns	recognized graph traversal, then BFS-for-shortest
4 Planning	stated the 3-step plan aloud, asked for buy-in
5 Driving	precise codebase-aware prompts; caught over-engineering; one-reprompt rule
6 Verification	compiled after each step; hand-traced; hunted edge cases
7 Communication	narrated every decision and <i>why</i> ; forward-looking wrap

Remember this

The winning session is a visible loop: **orient** → **recognize the pattern** → **plan aloud** → **drive the AI with precise prompts** → **verify every step** → **narrate the judgment**. The code was ordinary; the *control over the process* is what passed. Rehearse this rhythm until it's automatic.

The whole guide in three sentences

1. AI-enabled rounds test judgment, not typing — you're dropped into a real codebase with an AI and graded on whether you can direct it well. **2.** The skills are orientation (understand before prompting), pattern recognition (name the algorithm), planning (decompose into explainable phases), driving (precise prompts, delegate volume/keep judgment, one-reprompt rule), and verification (plausible ≠ correct — hunt the edge cases). **3.** None of it counts unless it's *visible*, so narrate your decisions — the interviewer is deciding whether they'd want you driving AI tools on their team.